

Data Link Protocol

(1st Lab Work)

FEUP

Computer Networks

2026/2027

- Goals

- » Implement a data link layer protocol, according to the specification provided in this document
 - this protocol implements transmitter and receiver functionality to transfer a file stored on a computer hard disk between computers connected through a RS-232 serial cable
- » Develop a simple transmitter and receiver data transfer application to test the protocol, according to the specification provided in this document
 - the application to be developed uses/invokes the functions implemented by the data link layer protocol
 - the data link layer protocol thus offers/exposes an API to the upper layer

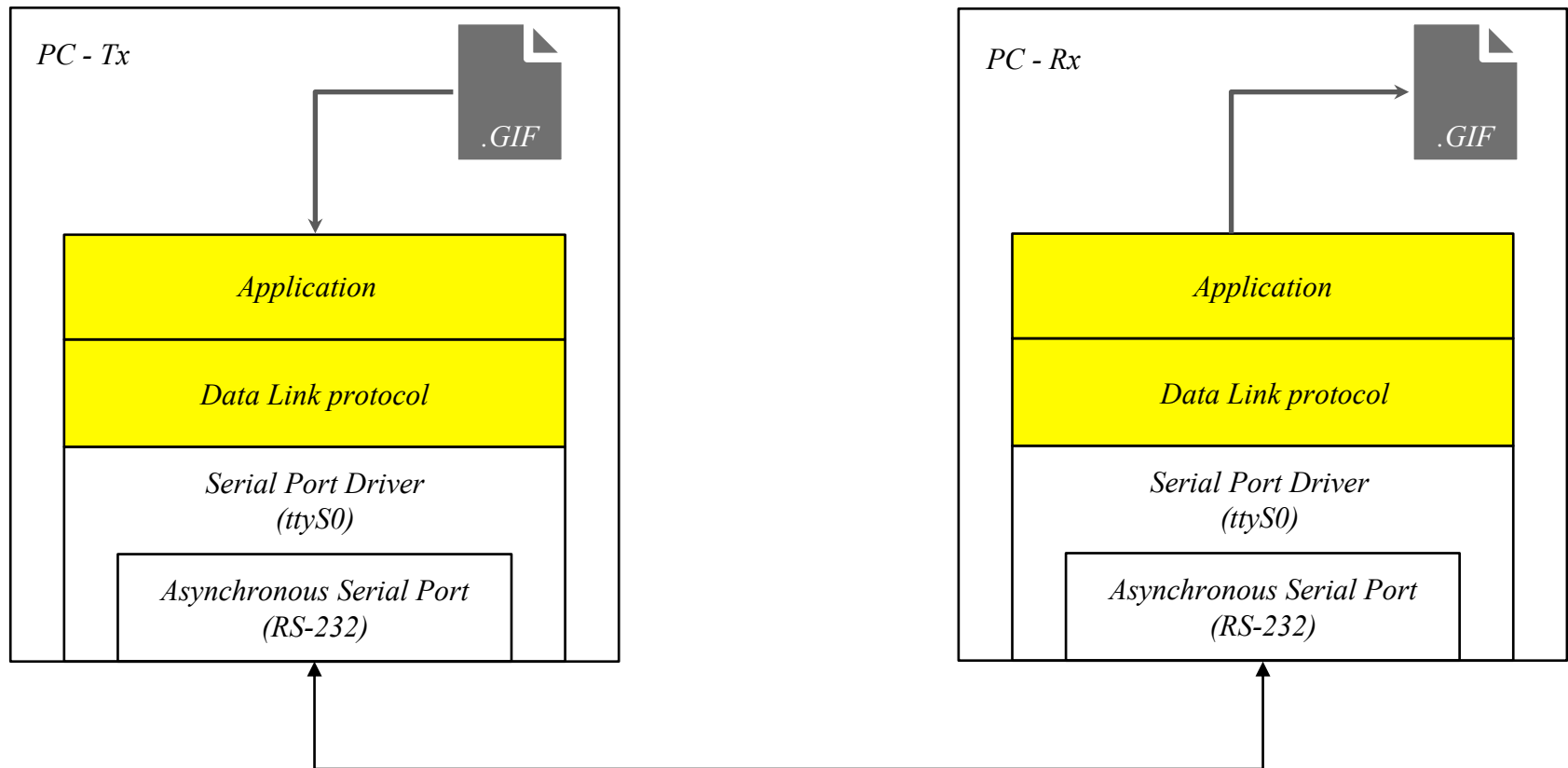
- Development environment

- » PC running LINUX
- » Programming language – C
- » Serial port RS-232 (asynchronous communication)

- Organization
 - » Groups of 2 elements
 - » Each group develops the transmitter and receiver
- Evaluation criteria
 - » Participation during class (continuous evaluation)
 - » Presentation and demonstration of the work on 5 milestones
 - M1: Exchange strings over serial connections;
 - M2: Sending and receiving control frame (SET/UA) and state machine in llOpenTx() & llOpenRx();
 - M3: Implement the Stop & Wait protocol in llSend() and llReceive();
 - M4: Timer and retransmission;
 - M5: Application layer implementation and correct API operation of llOpenTx(), llOpenRx(), llSend(), llReceive(), llCloseTx(), llCloseRx();
 - » Individual 15-minute quiz to be answered in the classroom, on the last class before the presentation
 - » Final report

Test configuration

4



Data Link Protocol - Goal and General Functionality

- Goal of the Data Link Layer Protocol
 - » Provide reliable communication between two systems connected by a communication medium – in this case, a serial cable
- Generic functions of data link protocols
 - » Framing
 - » Packaging and synchronisation/delimitation
 - » Connection establishment and termination
 - » Frame numbering
 - » Acknowledgement
 - » Error control (e.g.: Stop-and-Wait, Go-back-N, Selective Repeat)
 - » Flow control

Data Link Protocol - Functionality - Framing

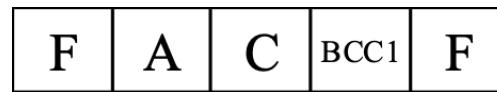
6

- » Packaging - data coming from the upper layer is packed into frames
 - » frames have an header, a body and a trailer
 - » user/application data goes into the body section
 - » these frames are designated as information frames
- » Frame synchronisation (delimitation)
 - » start and end of frames are uniquely identified so that data reception can become synchronised
 - » main alternative is to use a special character/flag at the beginning and end of frames
 - » need to make sure that its value does not occur elsewhere inside the frame
 - » transparency or stuffing mechanism (explained in slides [16](#) and [17](#))
 - » the size of frames may be implicitly determined
 - » by counting the number of bytes in between synchronisation flags
 - » or explicitly indicated
 - » in one field of the header

- » Connection establishment and termination
 - » exchange of specific messages sent in fixed-length frames
 - » designated of supervision frames having only control fields (no user data)
- » Frame numbering
 - » a counter module-n in the header of frames to allow to verify the correct sequence of information frames and/or the occurrence of duplicates
- » Positive Acknowledgement
 - » every time a frame is received without errors and in the right sequence a positive acknowledgement is sent back to the sender
- » Error control (e.g.: Stop-and-Wait, Go-back-N, Selective Repeat)
 - » use of timers (time-out) to enable re-transmission of un-acknowledged frames
 - » use of negative acknowledgement to request the retransmission of out-of-sequence or errored frames
 - » verification of duplicates which may occur due to re-transmissions

- The protocol to implement combines characteristics of existing real-world data link protocols
 - » agnostic to the type of user data to be transferred (independence and transparency)
 - » transmission organised into frames, which can be of three types
 - » Information (I), Supervision (S) and Unnumbered (U)
 - » Frames have a header with a common format
 - » only Information frames have a field for user data transport
 - » a field to transport a packet generated by the application, which content is not processed by the data link protocol
 - » Frame delimiting is done by means of a special eight-bit sequence (flag) and a byte stuffing technique ensures that this value will not occur inside the frame (explained in slide [17](#))
 - » The frames are protected by an error detection code
 - In frames S and U there is simple frame protection (since they do not carry data)
 - In I frames there is double and independent protection of the header and the data field (which allows to use a valid header, even if an error occurs in the data field)
 - » The Stop and Wait variant is used (unit window and modulo 2 numbering)

» Supervision (S) and Unnumbered (U) Frames



F	Flag		
A	Address Field		
C	Control Field to indicate the type of supervision frame/message		
	SET (set up)	0 0 0 0 0 0 1 1	
	DISC (disconnect)	0 0 0 0 1 0 1 1	
	UA (unnumbered acknowledgment)	0 0 0 0 0 1 1 1	
	RR (receiver ready / positive ACK)	1 0 1 0 1 0 1 R	R = N(r)
	REJ (reject / negative ACK)	0 1 0 1 0 1 0 R	R = N(r)
BCC₁	Protection Field to detect the occurrence of errors in header		

Format and types of frames

10

» Supervision (S) and Unnumbered (U) Frames

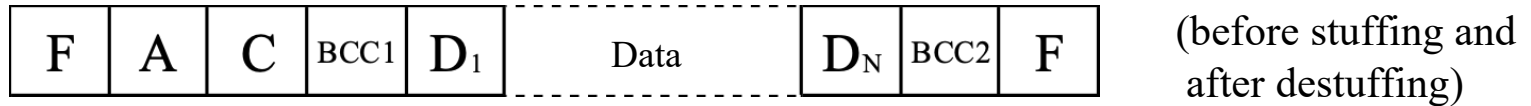
F	A	C	BCC1	F
---	---	---	------	---

Field	Value	Meaning
F	01111110 / 0x7E	Synchronisation: start or end of frame
A	00000011 / 0x03	Address field in frames that are commands sent by the Transmitter or replies sent by the Receiver
	00000001 / 0x01	Address field in frames that are commands sent by the Receiver or replies sent by the Transmitter
C	00000011 / 0x03	SET frame: sent by the transmitter to initiate a connection
	00000111 / 0x07	UA frame: confirmation to the reception of a valid supervision frame
	10101010 / 0xAA	RR0 frame: indication sent by the Receiver that it is ready to receive an information frame number 0
	10101011 / 0xAB	RR1 frame: indication sent by the Receiver that it is ready to receive an information frame number 1
	01010100 / 0x54	REJ0 frame: indication sent by the Receiver that it rejects an information frame number 0 (detected an error)
	01010101 / 0x55	REJ1 frame: indication sent by the Receiver that it rejects an information frame number 1 (detected an error)
	00001011 / 0x0B	DISC frame to indicate the termination of a connection
BCC1	A XOR C	Field to detect the occurrence of errors in the header

Format and types of frames

11

» Information Frames (I)



F **Flag**

A **Address Field**

C **Control Field to allow numbering information frames**

D₁ ... D_N **Information Field (packet generated by the Application)**

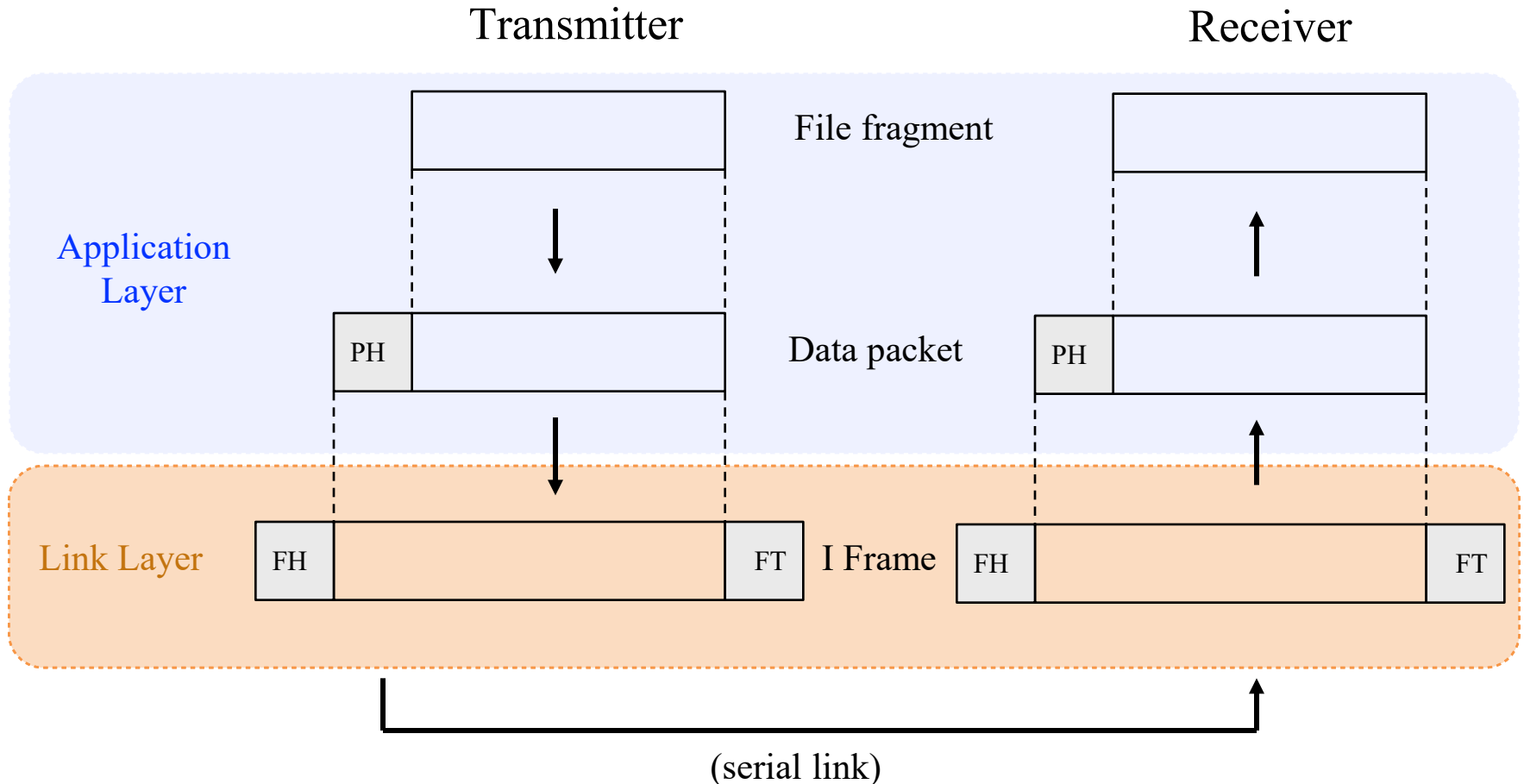
BCC_{1,2} **Independent Protection Fields (1 – header, 2 – data)**

S 0 0 0 0 0 0 0 S = N(s)

N(s) implements a module-2 counter, enabling to distinguish frame 0 and frame 1 successively throughout transmission

Field	Value	Meaning
F	01111110 / 0x7E	Synchronisation: start or end of frame
A	00000011 / 0x03	Address field in frames that are commands sent by the Transmitter or replies sent by the Receiver
	00000001 / 0x01	Address field in frames that are commands sent by the Receiver or replies sent by the Transmitter
C	00000000 / 0x00	Information frame number 0
	10000000 / 0x80	Information frame number 1
BCC1	A XOR C	Field to detect the occurrence of errors in the header
BCC2	D ₁ XOR D ₂ XOR D ₃ ... XOR D _N	Field to detect the occurrence of errors in the data field

- » At the application layer
 - » the file to be transmitted is fragmented - the fragments are encapsulated in data packets which are passed to the link layer one by one
 - » in addition to data packets (which contain file fragments), the Application protocol uses control packets
 - » the format of the packets (data and control) is defined ahead (slide [27](#))
- » At the link layer each packet (data or control) is carried in the data field of an I frame
- » The Transmitter is the machine that sends the file and the Receiver is the machine that receives the file
 - » thus, only the Transmitter transmits packets (data or control) and therefore only the Transmitter transmits I frames
- » Both the Transmitter and the Receiver send and receive frames (write or read frames into/from the serial line)



PH – Packet Header

FH – Frame Header

FT – Frame Trailer

Control packets are also transported in I frames

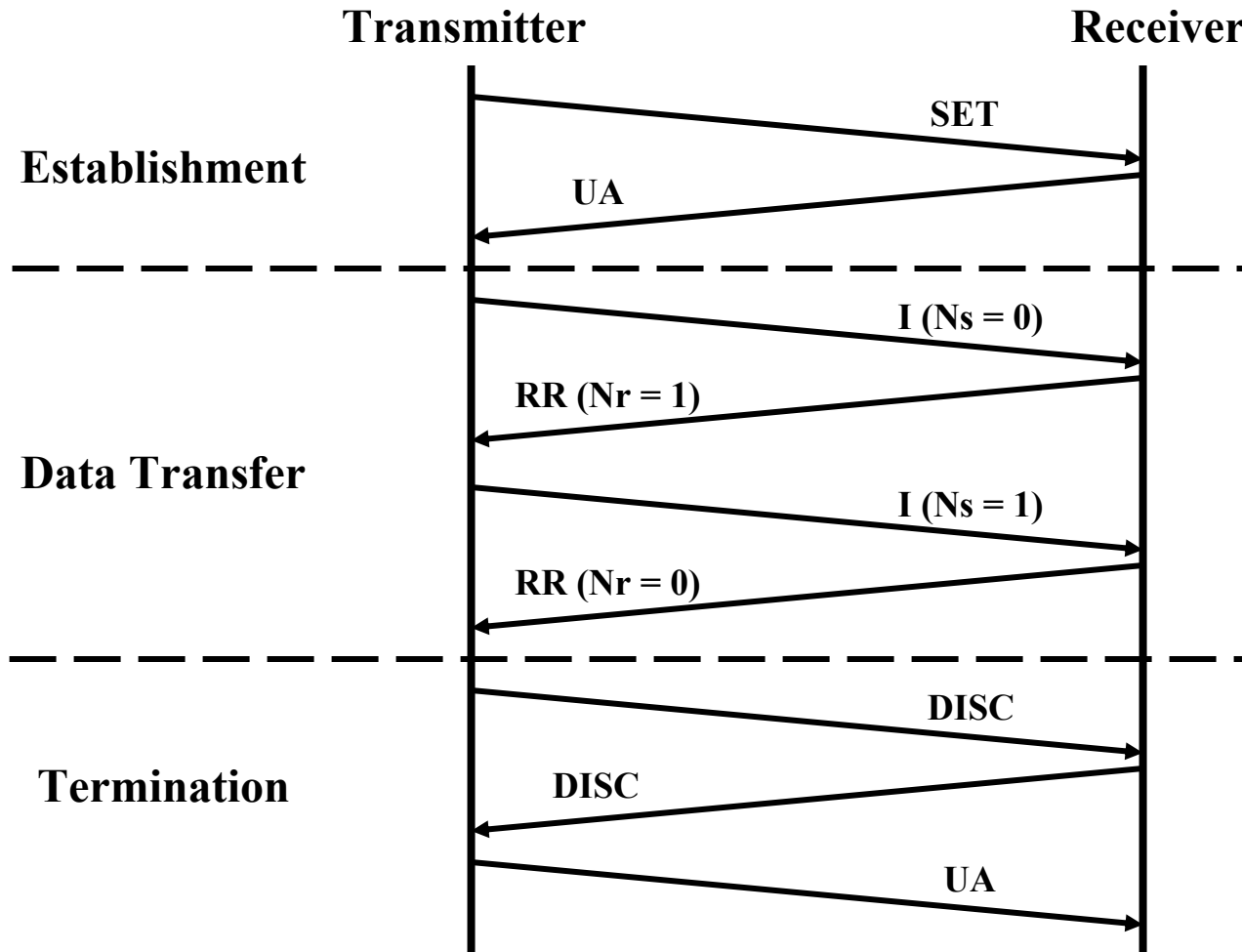
- » All frames are delimited by flags (**01111110**)
- » A frame can be started with one or more flags, which must be taken into account by the frame reception mechanism
- » Frames I, SET and DISC are designated Commands and the rest (UA, RR and REJ) are called Replies
- » Frames have a header with a common format
 - A (Address Field)
 - **00000011 (0x03)** in Commands sent by the Transmitter and Replies sent by the Receiver
 - **00000001 (0x01)** in Commands sent by the Receiver and Replies sent by the Transmitter
 - C (Control Field) – defines frame type and carries sequence numbers N(s) in I frames and N(r) in Supervision frames (RR, REJ)
 - BCC (Block Check Character) - error detection based on the generation of an octet (BCC) such that there is an even number of 1s in each position (bit), considering all octets protected by the BCC (header or data, as appropriate) and the BCC itself (before stuffing)

- » I, S or U frames with wrong header are ignored without any action
- » The data field of the I frames is protected by its own BCC (even parity on each bit of the data octets and the BCC)
- » I frames received with no errors detected in the header and data field are accepted for processing
 - If it is a new frame, the data field is accepted (and passed to the Application), and the frame must be confirmed with RR
 - If it is a duplicate, the data field is discarded, but the frame must be confirmed with RR
- » I frames with no header error detected but error detected (by the respective BCC) in the data field – the data field is discarded, but the control field can be used to trigger an appropriate action
 - If it is a new frame, it is convenient to make a retransmission request with REJ, which allows to anticipate the occurrence of time-out in the transmitter
 - If it is a duplicate, it must be confirmed with RR
- » I, SET and DISC frames are protected by a timer
 - In the event of a time-out, a maximum number of retransmission attempts must be made (the value must be configurable; for example, three)

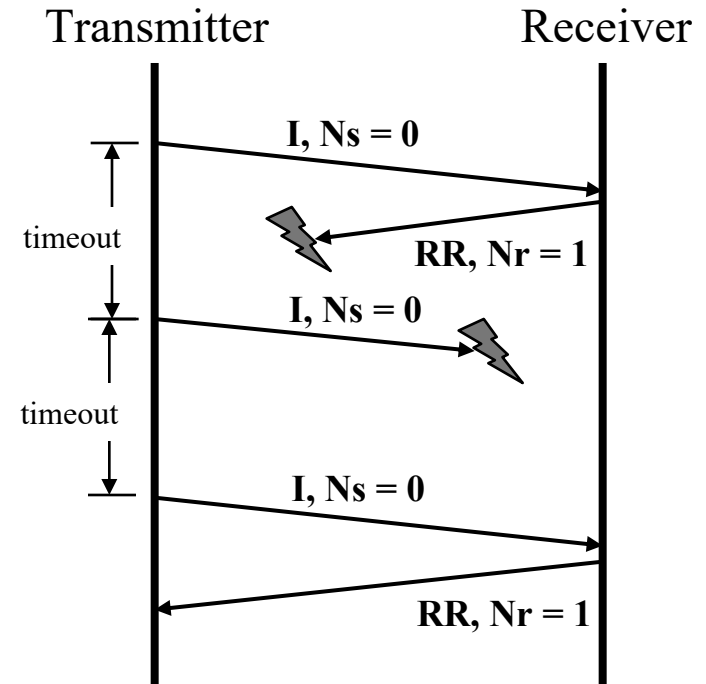
- » The transmission between the two computers is, in this work, based on a technique called asynchronous transmission
 - This technique is characterised by the transmission of "characters" (short string of bits, whose number can be configured) delimited by Start and Stop bits
 - Some protocols use characters (words) of a code (for example ASCII) to delimit and identify the fields that constitute the frames and to support the execution of the protocol mechanisms
 - In these protocols, the transmission of data transparently (regardless of the code used by the protocol) requires the use of escape mechanisms
- » The protocol to be implemented is not based on the use of any code, so the transmitted characters (consisting of 8 bits) must be interpreted as simple octets (bytes), and any of the 256 possible combinations can occur
- » To avoid the false recognition of a flag inside a frame, a mechanism that guarantees transparency is needed

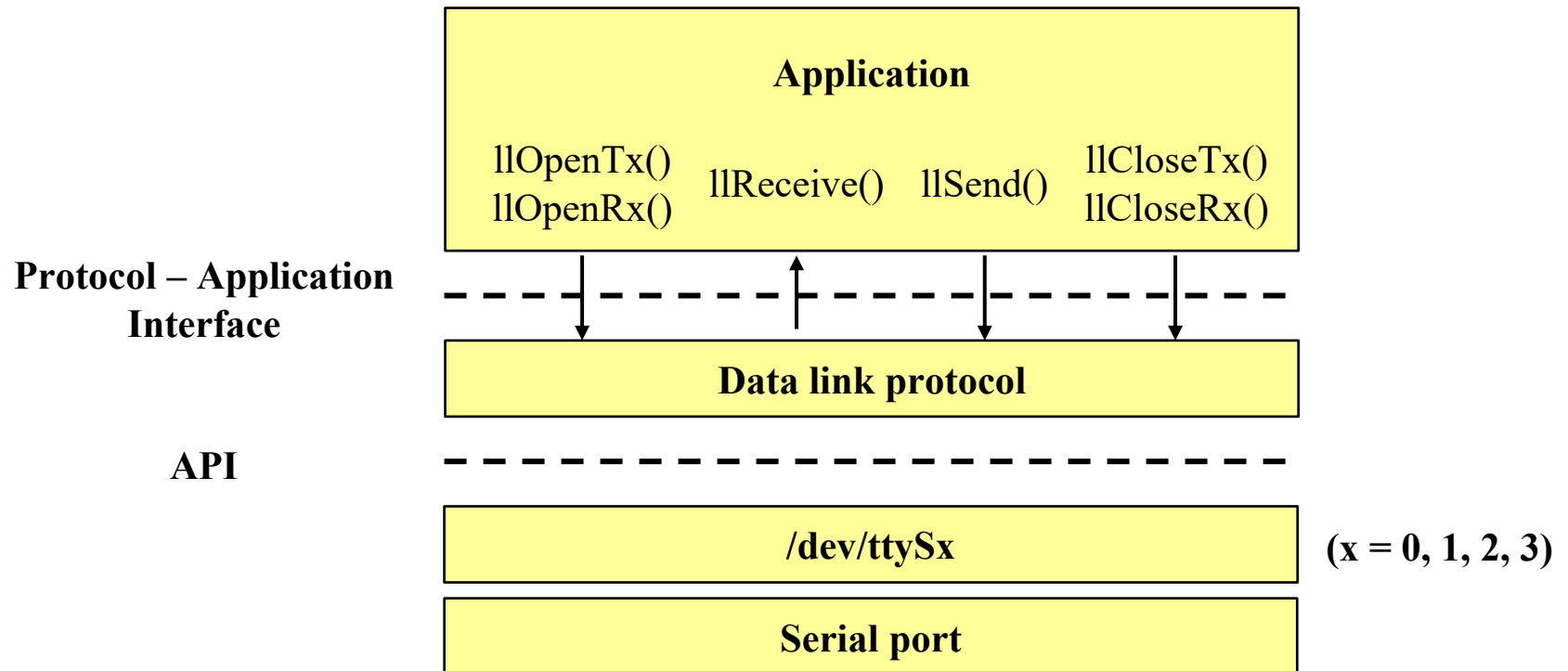
- » In the protocol to be implemented, the mechanism used in PPP is adopted, which uses the escape octet **01111101 (0x7D)**
 - If the octet **01111110 (0x7E)** occurs inside the frame, i.e., the pattern that corresponds to a flag, the octet is replaced by the sequence **0x7D 0x5E** (escape octet followed by the result of the exclusive or of the octet replaced with the octet 0x20)
 - If the octet **01111101 (0x7D)** occurs inside the frame, i.e., the pattern that corresponds to the escape octet, the octet is replaced by the sequence **0x7D 0x5D** (escape octet followed by the result of the exclusive or of the octet replaced with the octet **0x20**)
 - In the BCC generation, only the original octets are considered (before the stuffing operation), even if some octet (including the BCC itself) has to be replaced by the corresponding escape sequence
 - The verification of the BCC is carried out in relation to the original octets, i.e., after the inverse operation (destuffing) has been performed, if the replacement of any of the special octets by the corresponding escape sequence has occurred

Example of a typical frame sequence (without errors)



- Acknowledgement / Error Control
 - » Stop-and-Wait
- Timer
 - » Enabled after an I, SET or DISC frame
 - » Disabled after a valid acknowledgement
 - » If acceded (timeout), forces retransmission
- I frames retransmission
 - » After a time-out, due to loss of the I frame or its acknowledgement
 - Maximum number of predefined (configured) retransmission attempts
 - » After a receiving a negative acknowledgement (REJ)
- Frame protection
 - » Generation and verification of the protection fields (BCC)





- Link layer parameters structure

```
typedef struct {  
    char serialPort[50];           // Device /dev/ttySx (x = {0,1})  
    int baudRate;                  // Speed of the transmission  
    int timeout;                   // Retransmission timeout value in seconds  
    int nRetransmissions;          // Number of retries in case of failure  
} LinkLayer;
```

- Link layer functions

```
int llOpenTx(LinkLayer llParameters);  
int llOpenRx(LinkLayer llParameters);  
int llSend(const unsigned char *buf, int bufSize);  
int llReceive(unsigned char *buf);  
int llCloseTx();  
int llCloseRx();
```

Data link protocol interfaces – open

22

int llOpenTx(LinkLayer llParameters)

arguments

- llParameters: Link layer parameters

return

- 0 in case of success
- Negative value in case of error

int llOpenRx(LinkLayer llParameters)

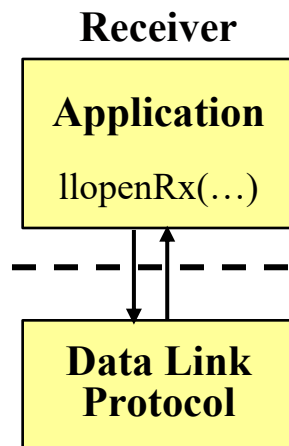
arguments

- llParameters: Link layer parameters

return

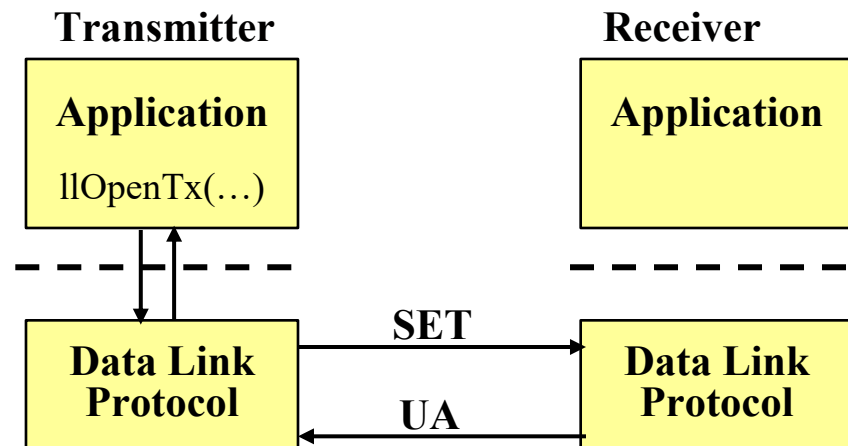
- 0 in case of success
- Negative value in case of error

Step I: the Rx application layer invokes
llOpenRx(...)



(I)

Step II: the Tx application layer invokes llOpenTx(...) that
runs at the link layer, exchanging supervision frames



(II)

Data link protocol interfaces – send / receive

23

int llSend(const unsigned char *buf, int bufSize) int llReceive(unsigned char *packet)

arguments

- buf: Array of bytes to transmit
- bufSize: Number of bytes to send (buffer size)

return

- Number of written bytes
- Negative value in case of error

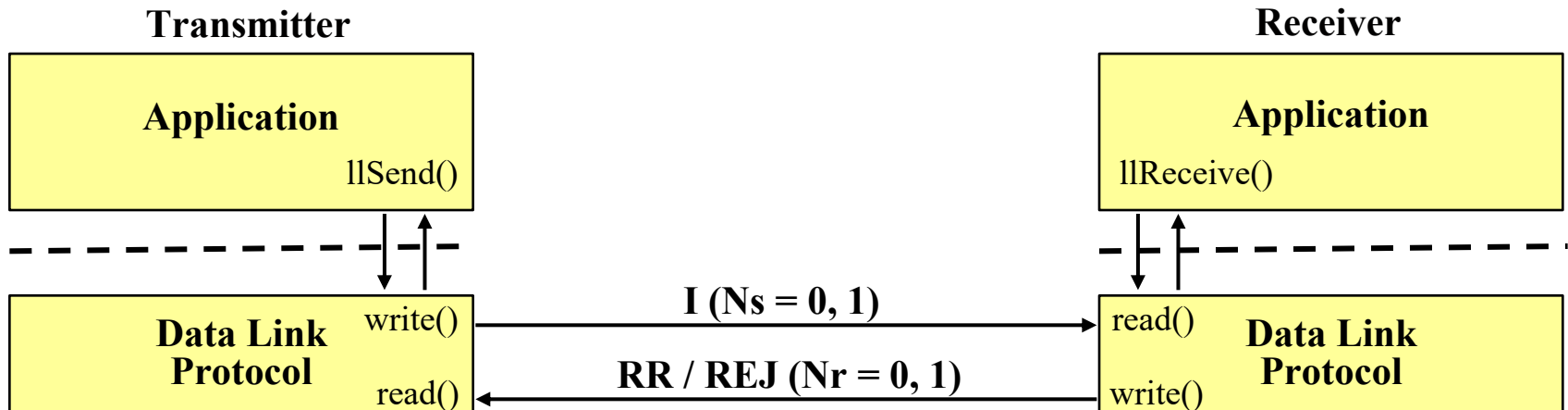
arguments

- packet: Array of bytes read

return

- Array length (number of bytes read)
- Negative value in case of error

The Tx application layer forms a packet (data or control) and invokes llSend(...); the Rx application layer invokes llReceive(...); llSend(...) and llReceive(...) exchange I and S frames. When frames are correctly received, both functions return the control to the application layer.



Data link protocol interfaces – close

24

int llCloseTx()

return

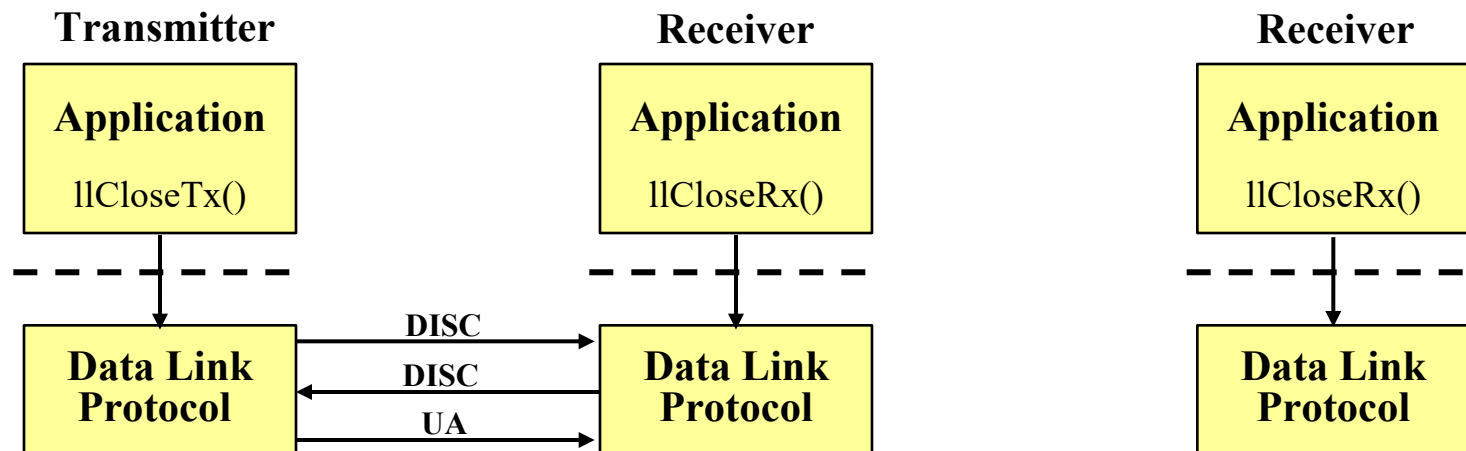
- 0 in case of success
- Negative value in case of error

int llCloseRx()

return

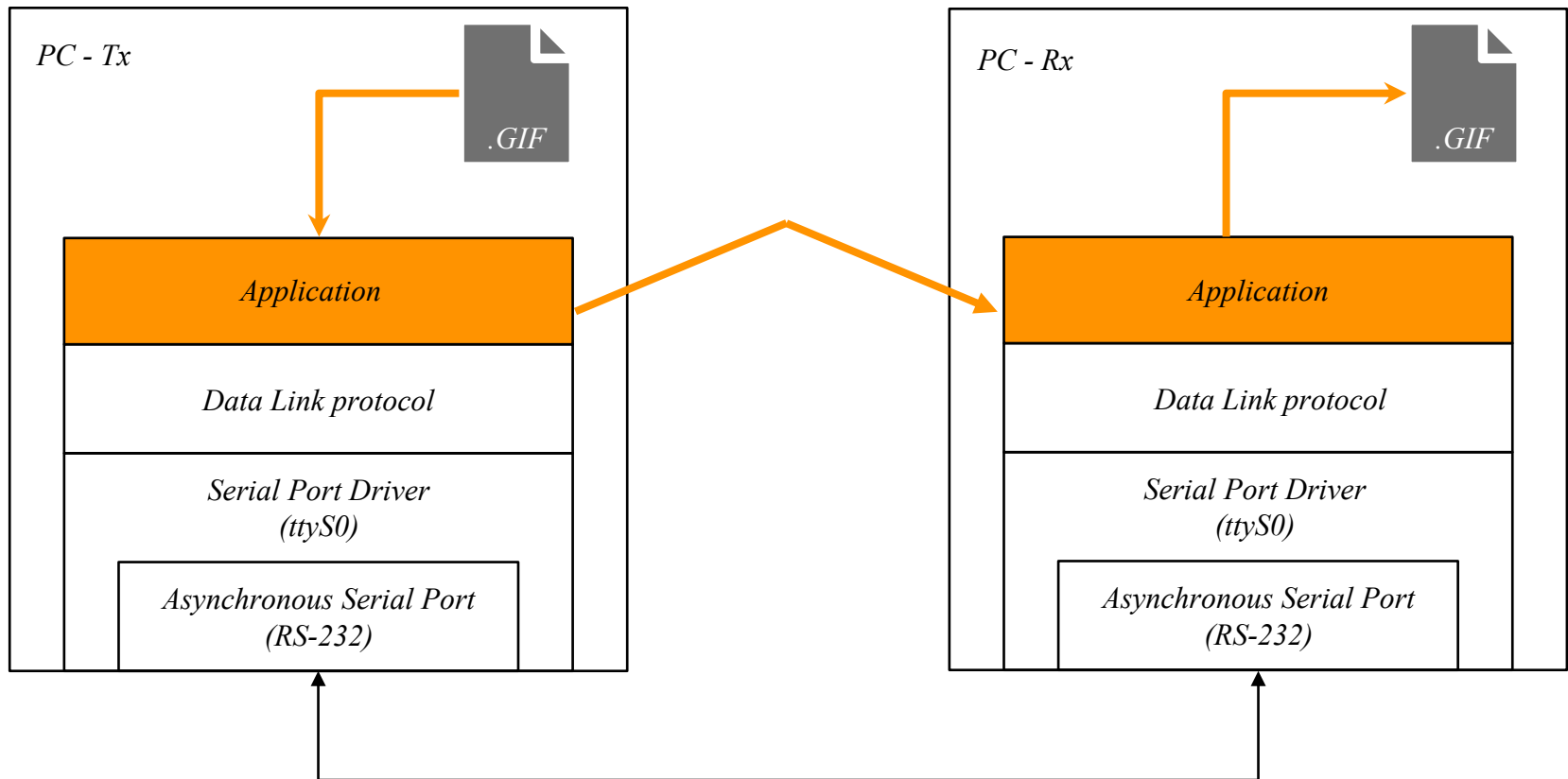
- 0 in case of success
- Negative value in case of error

The Tx and Rx application layers invoke llCloseTx() and llCloseRx(), respectively, which run at the link layer exchanging appropriate S frames



- The project code contains a library that implements the serial port layer (serial_port.c and serial_port.h)
- API
 - » int openSerialPort(const char *serialPort, int baudRate);
 - Open serial port with given baudrate
 - Returns > 0 on success or -1 on error
 - » int closeSerialPort();
 - Close serial port. Return 0 on success or -1 on error.
 - » int readByteSerialPort(unsigned char *byte);
 - Read 1 byte from the serial port.
 - Return number of bytes read (0 or 1) or -1 on error.
 - » int writeBytesSerialPort(const unsigned char *bytes, int nBytes);
 - Write nBytes from the bytes array to the serial port.
 - Return number of bytes written or -1 on error.

Simple application to access/store the file and pack/unpack data into packets



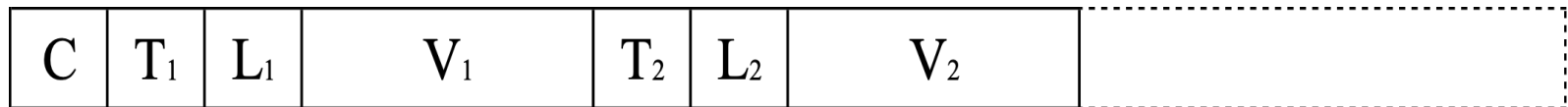
- The goal is to develop a very simple application protocol for transferring a file, using the reliable service offered by the data link protocol
- The application must support two types of packets sent by the Transmitter
 - » Control packages to signal the start and end of file transfer
 - » Data packets containing fragments of the file to be transmitted
- The control package that signals the beginning of the transmission (START) must have a field with the file size and optionally a field with the file name (and possibly other fields)
- The control packet that signals the end of transmission (END) shall repeat the information contained in the START packet
- Data packets must contain a field (two octets) that indicates the size of the respective data field ($D_1 \dots D_N$) to allow for additional checks regarding data integrity
 - » This size depends on the maximum size that may be established for the Information field of frames I

- Data packet



- » C – control field (value: 2 – data)
- » L₂ L₁ – number of octets (K) in the data field
- » P₁ ... P_K – packet data field (K octets) $(K = 256 * L_2 + L_1)$

- Control packet



- » C – control field (values: 1 – start; 3 – end)
- » Each parameter (size, file name or other) is coded as TLV (Type, Length, Value)
 - T (one octet) – indicates the parameter (0 – file size, 1 – file name, other values – to be defined, if necessary)
 - L (one octet) – indicates the V field size in octets (parameter value)
 - V (number of octets indicated in L) – parameter value

- Layered architectures are based on the principle of independence between layers.
- This principle has the following consequences in the scope of this work:
 - » At the data link layer
 - » no processing shall be done that may affect the header of packets passed by the application layer (to be transported in Information frames)
 - » this information is considered inaccessible to the data link protocol
 - » no distinction is made between control and data packets (all packets coming from the application layer are equally considered as user data)
 - » At the application layer
 - » there is no knowledge about the details of the data link protocol, only how its services can be accessed
 - » no knowledge about the structure of the frames and the respective delineation/synchronisation mechanism, the existence of stuffing (and which option is adopted), the protection mechanism of the frames, the numbering of frames, any eventual retransmissions of I frames, etc.
 - » all these functions are exclusively performed in the data link layer

- Data Link Layer
 - » Supervision frames (SET, UA, DISC, RR, REJ)
 - » Information frames (I)
 - » Retransmission of lost frames
 - » Byte stuffing / destuffing
 - » Detection of duplicate frames
 - » BCC1 calculation and check
 - » BCC2 calculation and check
 - » Reject frames (REJ)
 - » Protocol statistics

- Application Layer
 - » Control packets
 - » Data packets

- Statistical characterization of efficiency S (FER, a). Suggestions:
 - » 1) vary FER, T_{prop} , C , size of I frame (C = link capacity, bit/s)
 - » 2) measure the obtained transference time $S = R/C$ (R =received bitrate, bit/s)
 - » 3) plot S (FER, a) and check the validity of the known formulas for efficiency
 - » 4) repeat measurements
- To vary FER: random error generation on Information frames
 - » Suggestion – for each I frame correctly received, simulate (at the receiver) the occurrence of a header error and on the data field with pre-defined (and independent) probabilities, and proceed as a normal error
- To vary T_{prop} : generation of a simulated propagation delay
 - » Suggestion – use `alarm_sigaction.c` to include a processing delay on each received frame

Note: you can use `cable.c` to perform the protocol efficiency evaluation.

- Data link protocol
 - » Frame synchronisation process
 - » Retransmission process
 - » Robustness to errors
 - » Error control
- Application protocol
 - » Control packets
 - » File integrity
- Code organisation
 - » Interface between layers (functions)
 - » Layer independency
- Statistical characterisation of the protocol efficiency
- Demonstration and report
- Penalties
 - » Delays on the demo and / or report submission